

Phospho Site Localization via Per-Scan Isomer Competition

Implementation plan for ‘Idea 1’ — turning deconvolution weights into a localization call + confidence

Pioneer.jl — prepared for N. Wamsley

2026-07-03

1. What we are building and why

Observation (measured). On the Coon 225-standard ground-truth benchmark, choosing the phospho site by **highest deconvolution weight** instead of best q-value cut the false-localization rate from **23.2% -> 5.6%** — using a number Pioneer already computes. The deconvolution weight is the *positional-isomer competition*: the solver splits a co-isolated signal between isomers, and the split is driven by the site-determining ions (weighted by Prosit-PTM predicted intensities).

Goal. Promote that signal from a post-hoc analysis trick into a first-class, per-scan mechanism inside the main search that:

1. produces a **localized site** per identified phosphopeptide,
2. attaches a **localization confidence** (a weight fraction) that can be **filtered at a cutoff** (the field’s `PTM.SiteProbability` ≥ 0.75 analogue), and
3. **does not discard co-present isomers** — two real isoforms that resolve chromatographically each survive at their own apex; only genuinely ambiguous cases are flagged.

Success criteria

- **FLR** on the standards harness drops toward the paper’s **1-5%** at a reasonable confidence cutoff (baselines: best-q 23.2%, raw weight 5.6%), with **ID recovery held** (currently 93%).
- A per-precursor `localization_confidence + localization_ambiguous` output that is **monotonic vs. true FLR** (higher confidence -> lower error), so the cutoff is meaningful.
- **Co-present resolved isomer pairs** (there are 3 in the benchmark) both survive.
- **Negligible search-time cost** (the mechanism is an $O(n)$ per-scan sweep).

2. Where this lives in the code (grounded)

The ingredients already exist:

- **Per-scan weight, per PSM.** The main-search deconvolution (`MainSearch/deconvolution.jl`, `process_scans_fused.jl`) emits one weight per (`scan_idx`, `precursor_idx`) PSM. PSM rows are **contiguous by scan_idx** (natural deconv emit order).
- **A per-scan competition sweep already runs.** `add_scan_competition_features!` (`MainSearch/features.jl:1528`) does an $O(n)$ contiguous-by-scan sweep and writes

`weight_ratio_at_scan = weight/max(weight)` and `weight_rank_at_scan` — but ranked over all PSMs at the scan. **This is our template and insertion point.**

- **Precursor -> peptide identity** via `LibraryPrecursors` (`getSequence`, `getStructuralMods`, `getPrecCharge`, `getMz` in `precursors.jl`).

The sibling-isomer key (corrected)

Positional isomers are **NOT** grouped by `base_pep_id` (verified: each isomer gets its own id). They are grouped by (**sequence, precursor charge, precursor m/z**) — same peptide, same phospho count (hence same isolated m/z), different position.

For now, group directly on (sequence, charge, round(m/z, ppm)) looked up per `precursor_idx` from the library — no build/format change, correct immediately. **Eventually** the library could carry a precomputed `isomer_group_id` so the grouping is a single integer read instead of a string-tuple compare in the hot path — a *speed* optimization, not required for correctness. We start with the direct key.

3. The algorithm

3.1 Per-scan sibling weight fraction (the core signal)

At each scan, among the co-isolated **sibling isomers** (same `isomer_group_id`), for each PSM compute:

$$f_i = \frac{w_i}{\sum_{j \in \text{siblings at this scan}} w_j}$$

`f` in $(0,1]$ is the fraction of the group’s deconvolved signal this isomer captured **at this scan** — a direct, intensity-aware localization vote. (Singleton groups get `f` = 1.)

Critical: normalize over the sibling isomers only — not over all PSMs at the scan. The existing `weight_ratio_at_scan` divides by the max weight across *everything* that matched the scan, so a co-isolated unrelated peptide with high weight would dilute the fraction and destroy the localization signal. We do **not** reuse that value; we compute a fresh sum restricted to the isomer group. The denominator is $\sum$ over siblings, so within a group the fractions sum to 1 — this is what makes `f` read as a site probability (the `>= 0.75` analogue).

3.2 Per-precursor localization call + confidence

A precursor’s PSMs span its elution. Define its **apex** as its max-weight scan (or, better, the peak-integrated version). Then:

- **Localized site** = the isomer that wins its group at the apex.
- **Localization confidence** = `f` at the apex (Phase A), upgraded to the **peak-integrated** fraction ($\sum_{\text{apex} \pm \delta} w_i$) / ($\sum_{\text{apex} \pm \delta} \sum_j w_j$) (Phase B) — steadier than a single scan.

3.3 Competition / survival (the filter, and the co-presence guardrail)

Operate per sibling group across all its scans:

- An isomer **survives** if it **dominates** (`f >= margin`, e.g. 0.75, or is the group max with `f_best / f_2nd >= ratio`) at **at least one** scan — i.e. it has its **own apex**.

- **Co-present, resolved** -> siblings A and B dominate at **different** scans (different RT) -> both survive (nothing discarded). *This is the answer to “what if both are truly present.”*
- **Spurious split** -> an isomer that never dominates anywhere (only ever a weak share of a real isomer’s signal) -> **suppressed** (no PSM -> no ID).
- **Genuinely ambiguous** -> at the shared apex no isomer dominates (adjacent/co-eluting sites, e.g. the benchmark’s ADEPSSEESDLEIDK S5-vs-S6 at equal weight) -> **keep the group’s best PSM and set `localization_ambiguous = true`**. Never delete both.

3.4 Output

Add to the precursor output: `localized_mods` (winning isomer), `localization_confidence` (the weight fraction), `localization_ambiguous`. Downstream filtering/reporting applies a **cutoff** on `localization_confidence` (config-driven), exactly like Spectronaut’s 0.75.

4. Implementation steps (phased, each validated on the harness)

Phase A — reporting only (safe; no PSMs dropped). Prove the value and calibrate the cutoff before changing what gets identified.

1. `isomer_group_id`: precompute at library load (`loadSpectralLibrary.jl`), grouping precursors by (`sequence`, `charge`, `round(mz)`); expose `getIsomerGroupId(precursors, idx)`.
2. `add_isomer_competition_features!`: clone `add_scan_competition_features!`’s contiguous-by-scan sweep, but within each scan **sub-group by `isomer_group_id`** and emit `iso_weight_fraction_at_scan` (and `iso_weight_rank`). Same $O(n)$ structure -> negligible cost.
3. Aggregate per precursor to a `localization_confidence` (apex `f`) and carry it through Main-Search/ScoringSearch output columns.
4. **Validate**: re-run the FLR harness; plot **FLR vs confidence cutoff** and **ID retained vs cutoff**. Expect FLR falling toward ~1-5% as the cutoff rises. This alone ships a usable localization score.

Phase B — filtering + peak integration.

5. Switch confidence to the **peak-integrated** fraction (apex +/- a few scans).
6. Apply the **survival filter** (3.3): suppress isomers that never dominate; flag ambiguous groups. Config-gate it (`optimization.localization.enabled`, `margin`, `min_confidence`).
7. **Validate**: FLR + ID recovery on the harness; confirm the 3 co-present isomer pairs both survive; confirm no double-deletion of ambiguous pairs.

Phase C — productize.

8. Config knobs (cutoff, margin), output columns (`localized_mods`, `localization_confidence`, `localization_ambiguous`), docs. Optional: a per-site probability (distribute group confidence across the winning isomer’s site-determining evidence).

5. Validation plan (the harness we already have)

- **Primary**: the ground-truth standards harness (`analyze_flr2.jl` + `coon_225_standards.tsv`), reported as **FLR and ID-recovery vs. confidence cutoff**. Baselines to beat: best-q 23.2%, raw weight 5.6%; target: $\leq 5\%$ at a cutoff that retains most IDs.
- **Robustness**: repeat across the **3 reps** (is 5.6% stable?) and down the **dilution series** (2500 -> 39 amol) to get the **FLR-vs-abundance** curve the paper reports.

- **Ground-truth-free cross-check:** the **phospho-proline** / **decoy-amino-acid** FLR estimator (allow phospho on P or A/L; any hit there is false) — so users can measure FLR on their own data. Cheap to add and validates the cutoff independent of the synthetic standards.
- **No-harm:** confirm non-phospho searches (MTAC yeast) are unchanged when localization is off.

6. Risks & subtleties

- **Sibling key correctness** — (`sequence`, `charge`, `mz`), not `base_pep_id`; include target/decoy status so decoys don't pollute a group. Verify with the benchmark's known isomer sets.
- **Deconvolution conditioning** — sibling columns are collinear (share all but site-determining ions); the split is regularization-sensitive. The earlier **L2 experiment** (FLR modulated with ridge) suggests a **small L2 may stabilize the fraction**; worth a joint sweep (localization confidence x deconvolution reg).
- **Apex definition** — single scan is noisy; peak-integration (Phase B) is the fix. Adjacent-site co-elution stays genuinely ambiguous -> correctly flagged, not forced.
- **Cutoff calibration** — trades recall for FLR; calibrate on the harness + decoy-AA estimator, not by guesswork.
- **This is orthogonal to, and does not replace, Idea 2** — if a site-determining ion was never in the library top-10 (measured: ~2% of multi-STY standards, all adjacent-site), no per-scan competition can recover it. Idea 2 (guarantee site-determining fragments) remains a targeted follow-up for those cases.

7. Deliverables & sequencing

Phase	Deliverable	Gate
A	<code>isomer_group_id</code> + <code>add_isomer_competition_features</code> + <code>localization_confidence</code> output (reporting only)	FLR-vs-cutoff curve on the harness; FLR $\leq$ ~5% at a usable cutoff
B	peak-integrated confidence + survival filter + ambiguity flag	ID recovery held; 3 co-present pairs both survive; no double-deletion
C	config cutoff, output columns, docs, optional per-site probability + decoy-AA FLR estimator	reproducible across 3 reps + dilution series

Phase A is the high-value, low-risk first step: it ships a real, filterable localization score by *surfacing* the signal we've already validated, without changing which precursors are identified.